RestoNiti

Cloud Data Platform - Architecture Plan

Status: Proposed for architect review | 10 September 2026

Decision: Keep Azure for the API, keep Cloudflare R2 for encrypted backups (and later QR menu files), replace Azure SQL Database with Azure Database for PostgreSQL Flexible Server for metadata and future owner-dashboard projections.

Out of scope: rewriting the Electron POS, rewriting the cloud API in Go, moving to AWS or Oracle.

1. Decision requested

Layer	Choice
POS (source of truth)	Unchanged: local SQLite on the restaurant PC
Cloud API	Unchanged: .NET 10 Minimal API on Azure App Service (Linux)
Object storage	Unchanged: Cloudflare R2 (private bucket)
Relational store	CHANGE: Azure SQL -> PostgreSQL Flexible Server
QR view-only menu (later)	Cloudflare Pages + menu JSON on R2
Owner dashboard (later)	Same API + Postgres read models (not a live SQLite copy)

Please confirm or reject this direction before we migrate production metadata off SQL Server.

2. Context

2.1 Product constraints

RestoNiti is offline-first. Billing, KOT, and printing must work with no internet. SQLite on the PC is the system of record. Cloud must never block POS operations.

- Now: disaster recovery - encrypted .rnbak off-site; restore via restaurant code + owner recovery key.
- Later: owner dashboard of sales summaries, not the POS itself.
- Later: QR view-only menu (static page, no ordering in v1).

2.2 What already exists

Piece	Today
Desktop	Electron POS; local .rnbak (gzip + AES-256-GCM); cloud client + scheduler
Cloud API	services/cloud-api - activate, recover, upload-intent, content, verify, list
Metadata	Restaurants, Installations, RestaurantSecrets, BackupRecord (EF CloudDbContext)
Dev DB	SQLite file
Staging DB	Azure SQL Serverless Gen5 (expensive if left awake)
Blobs	Private R2; API buffers upload
App Service	Staging Windows; production target Linux B1

EF today: SQLite if CloudDb looks like a file path; otherwise UseSqlServer with retry for error 40613.

2.3 Cost problem

Database SKU	Compute / month
SQL Serverless 0.5 vCore left awake	~Rs. 20,000
SQL provisioned 1 vCore (honest dashboard SKU)	~Rs. 11,679
PostgreSQL Flexible B1ms always on	~Rs. 1,709

Full platform (API + DB + R2 + OTP), dashboard live	Rs./month	Break-even @ Rs. 10k Premium/yr
Linux B1 + SQL 1 vCore + R2	~14,700	~19 Premium outlets
Linux B1 + Postgres B1ms + R2	~4,800	~6 Premium outlets
Greenfield Go + Fly + Postgres + R2	~4,300	~5 Premium outlets

Saving vs SQL 1 vCore: ~Rs. 10,000/month (~Rs. 1.2 lakh/year). Almost all of it is SQL Server vs Postgres, not Azure vs AWS.

3. Why Azure + Postgres

3.1 Why stay on Azure for the API

- API is already written, deployed, and exercised (activate / upload / R2 / recover).
- App Service Linux B1 is a known cheap always-on SKU (~Rs. 1,255/month).
- Central India matches data-residency preference.
- Changing only the database provider is a bounded EF change.

3.2 Why Postgres instead of Azure SQL

- Current schema is simple keys/indexes; no T-SQL or SQL Agent dependency.
- EF Core Npgsql is first-class; local tests stay on SQLite.
- Owner dashboard cannot rely on SQL Serverless auto-pause. Provisioned SQL 1 vCore is ~7x B1ms.
- Postgres is the natural store for later owner read-models (event projections).

3.3 Why keep Cloudflare R2

- Large ciphertext blobs; R2 has no internet egress fee.
- POS already uses the AWS S3 SDK against R2.
- Later QR menus are small JSON on the same bucket + Pages.

3.4 Alternatives rejected

Option	Why not for production now
Keep SQL Serverless	High bills if awake; dashboard makes pause unreliable
Keep SQL 1 vCore	Correct SQL SKU; ~Rs. 10k/mo extra vs B1ms, no product gain
AWS RDS + App Runner	Similar/higher small-scale cost; re-platform before paying clients
Oracle Always Free / ATP	Idle-stop, 20 GB cap, poor .NET fit, expensive paid jump
Rewrite in Go + Fly	Extra save ~Rs. 500/mo; rewrite + re-test dominate
Firebase / Cosmos as POS truth	Violates offline-first SQLite
Backup bytes in Postgres	Wrong store; blobs stay on R2

3.5 Drawbacks we accept

- Migration: UseSqlServer -> UseNpgsql, connection string, retry, staging cutover.
- B1ms is burstable; heavy dashboard may need B2s (~Rs. 6.8k compute) - still below SQL 1 vCore.
- Different ops (pgAdmin, firewall, upgrades). Still on Azure billing/Entra.

4. Target architecture

```
Restaurant PC (Electron)
  SQLite  = source of truth for billing / KOT
  .rnbak  = gzip + AES-256-GCM (key in OS secure storage)
       |
       | HTTPS (never blocks POS)
       v
Azure App Service (Linux)  .NET 10 Cloud API
       |
       +-- PostgreSQL Flexible Server     metadata + later owner projections
       |
       +-- Cloudflare R2                  ciphertext backups, later menu JSON
              |
              +-- Pages  menu.restoniti.com/menu/{slug}     QR (later)
              +-- Pages  owner.restoniti.com                owner PWA (later)
```

Trust boundary: cloud stores ciphertext + metadata. Recovery key is not stored on the server (scrypt verifier + wrapped key only).

5. How we will use each component

5.1 PostgreSQL - what goes in

Data	Use
restaurants	Tenant id, public restaurant code
installations	Device, hashed installation token, app version
restaurant_secrets	Recovery-key verifier (scrypt), wrapped backup key - not the raw key
backups	Id, object key, checksum, size, status, timestamps, retention
Later events / rollups	Projected sales, voids, daily totals (restaurant_id on every row)

One shared database, restaurant_id isolation. Do not put SQLite files, .rnbak bodies, or menu images in Postgres.

5.2 R2 - what goes in

Object	Use
backups/{restaurantId}/{installationId}/{backupId}.rnbak	Encrypted DR (as today)
Later menus/{slug}.json	Published view-only menu for QR

5.3 App Service

- Authenticate installation Bearer tokens; activate / recover.
- Create backup records; buffer and PUT ciphertext to R2 (current reliable path).
- Later: owner OTP, projection ingest, owner read APIs.
- /health must not keep Postgres busy; split liveness vs readiness if needed.

5.4 Load by phase

Phase	Postgres	SKU intent
Now - DR only	Low QPS; daily uploads	Always-on B1ms once clients are live
Owner dashboard	Small continuous reads/writes	Watch CPU credits; scale to B2s if needed

Do not use Azure SQL Serverless as the long-term dashboard store.

5.5 POS contract unchanged

Same endpoints: activate, recover, upload-intent, content PUT/GET, verify, list. POS_CLOUD_API_URL / POS_CLOUD_BACKUP_ENABLED unchanged. No POS schema change.

Local/CI: SQLite when CloudDb is a file. Staging/production: Postgres.

6. Implementation plan

Phase 0 - Hygiene

- Production App Service Linux B1 (not Windows).
- Pause unused staging SQL; no chatty health+EnsureCreated loops.
- Secrets in App Service / Key Vault, not source control.

Phase 1 - Postgres for existing metadata (this decision)

- Add Npgsql.EntityFrameworkCore.PostgreSQL; UseNpgsql when not SQLite.
- Npgsql retry on transient errors.
- Provision Flexible Server Central India B1ms; SSL; firewall to App Service (or Private Endpoint).
- Replace EnsureCreated in production with EF migrations.
- Staging cutover + activate/upload/list/recover test; then decommission SQL.

Success: dotnet test + POS Backup Now -> R2 object + list/restore.

Phase 2-4

- Phase 2: Client DR only. Premium = Classic + cloud backup.
- Phase 3: POS event log -> API -> Postgres projections. No full SQLite upload loop. Owner PWA on Pages.
- Phase 4: QR view-only via R2 + Pages /menu/{slug}.

7. Security checklist

Topic	Approach
Backup confidentiality	Client-side AES-256-GCM; R2 holds ciphertext
Recovery	Owner-held key; server stores verifier + wrapped key only
Tenant isolation	Queries scoped by restaurantId from installation auth
Secrets	Hashed installation tokens; Key Vault / App Settings for R2 + Postgres
Network	Postgres firewall: App Service outbound or VNet
Region	Central India API + Postgres; R2 as already used
QR	Public read of menu only; no POS credentials

8. Production cost envelope (shared)

Resource	Target SKU	Rs./month
App Service	Linux B1	1,255
Postgres	Flexible B1ms + storage later	1,709 + storage
R2	Standard	100-300 early
OTP / domain / logs	Buffer	400-1,400
Full Premium platform		~4,800-5,500

Scale trigger: Postgres CPU/credits -> B2s or GP 1 vCore Postgres, not SQL Server.

9. Risks

Risk	Mitigation
EF/Npgsql differences	Integration tests on real Flexible Server before any client
B1ms throttle	Metrics + alert; pre-approved B2s
R2 large PUT	Keep API-buffered upload
EnsureCreated in prod	Replace with migrations
Staging cost relapse	Stop unused SQL; do not leave Serverless awake

10. Open questions for the architect

- Network: public + firewall vs Private Endpoint + App Service VNet?
- HA: single-AZ B1ms enough for v1 DR, or zone redundancy from day one?
- Approve EF migrations as the DDL source of truth?
- Key Vault mandatory for R2 + Postgres secrets in production?
- Accept B1ms burstable for 12 months, or start small GP?
- Is R2 (Cloudflare) acceptable alongside Azure Central India for ciphertext blobs?

11. Recommendation

Approve Azure App Service Linux + PostgreSQL Flexible Server + Cloudflare R2. Fits offline POS + encrypted DR + later owner projections. Saves ~Rs. 10k/month vs SQL 1 vCore without rewriting the API. Reject AWS/Oracle/Go rewrite now.

Ask: sign off Phase 1 (Npgsql + Flexible Server B1ms + Linux B1 + R2 as-is) and section 10 so engineering can migrate staging off Azure SQL.

Markdown source: docs/RESTONITI_AZURE_POSTGRES_ARCHITECTURE_PLAN.md